



UNIVERSIDAD CENTRAL DE VENEZUELA
FACULTAD DE CIENCIAS
ESCUELA DE COMPUTACIÓN
ADMINISTRACIÓN DE BASES DE DATOS



PROYECTO #1 — STREAMUCV
INFORME TÉCNICO DE LA SOLUCIÓN

Grupo Docente:

Profa. Concettina Di Vasta
Br. Joiner Rojas
Br. Maximiliano Barboza

Integrantes:

Ronald Herrera
Brhandon Palomo
Sebastián Russian

Caracas, Junio de 2026

Índice

1. Descripción General de la Solución del Proyecto	2
1.1. Arquitectura de la Solución	2
1.2. Beneficios del Diseño	2
2. Explicación del Uso del Diccionario de Datos	3
2.1. Listado de Tablas e Índices	3
2.2. Conteo de Tablas e Índices por Tabla	3
2.3. Restricciones del Esquema	3
2.4. Columnas de cada Índice	3
2.5. Triggers del Esquema	4
2.6. Tamaño Físico por Tabla	4
2.7. Estimación del Tamaño de Registro	4
2.8. Tamaño de cada Columna según su Tipo de Dato	5
2.9. Factor de Bloqueo de Tablas e Índices	5
2.10. Costo de Consultas de Igualdad	5
2.10.1. 1. Detección de Índices	5
2.10.2. 2. Caso A: Escaneo Completo (Full Table Scan)	6
2.10.3. 3. Caso B: Búsqueda por Índice (Index Seek)	6
3. Instrucciones de Ejecución	7
3.1. Paso 1: Prerrequisitos	7
3.2. Paso 2: Instalar Dependencias	7
3.3. Paso 3: Crear y Poblar la Base de Datos	7
3.4. Paso 4: Configurar la Conexión	8
3.5. Paso 5: Ejecutar la Aplicación	8
3.6. Paso 6: Navegación	8

1. Descripción General de la Solución del Proyecto

StreamUCV es una aplicación en **Python** con interfaz **Streamlit** que consulta en tiempo real el **Diccionario de Datos** de una instancia **SQL Server 2022** y presenta de forma visual los metadatos del esquema `streaming`.

1.1 Arquitectura de la Solución

La solución se organiza en capas lógicas con responsabilidades bien delimitadas, asegurando modularidad y mantenimiento sencillo:

main.py	Punto de entrada principal que inicializa Streamlit y lanza la aplicación.
app/config.py	Centraliza los parámetros de conexión (SERVER, DATABASE, etc.) y las constantes físicas de cálculo (páginas de 8 KB y velocidad de 17 MB/s).
app/connection.py	Administra el canal de comunicación con SQL Server mediante pyodbc, gestionando los intentos de conexión física.
app/queries/	Módulos individuales (r01.py a r10.py) que aíslan la lógica de cada requerimiento del diccionario de datos, retornando conjuntos de datos listos para consumir.
app/ui/streamlit_app.py	Define la interfaz gráfica, organizando las pestañas, selectores dinámicos y la barra de navegación lateral.

1.2 Beneficios del Diseño

La arquitectura de la solución aporta ventajas clave frente a una ejecución SQL de consola tradicional:

Portabilidad	Los parámetros de conexión pueden modificarse y probarse en tiempo de ejecución desde la interfaz web, permitiendo cambiar de servidor sin alterar el código.
Eficiencia	Las consultas se realizan sobre el Diccionario de Datos del motor (<code>sys.*</code>), evitando escaneos costosos sobre datos de usuario reales.
Interactividad	Transforma la administración pasiva de la base de datos en una experiencia visual enriquecida mediante tablas dinámicas, filtros reactivos y reportes limpios.

2. Explicación del Uso del Diccionario de Datos

El Diccionario de Datos (D/D) es el conjunto de vistas internas del esquema `sys` que almacenan los metadatos de SQL Server. La aplicación consulta exclusivamente estas vistas para obtener información sobre la estructura lógica y física de la base de datos.

2.1 Listado de Tablas e Índices

Lista las tablas e índices del esquema `streaming`.

- **Vistas utilizadas:** `sys.tables`, `sys.schemas`, `sys.indexes`.
- **Lógica:** Se unen las vistas filtrando por `s.name = 'streaming'`. Para índices se excluyen las filas con `i.name IS NULL (Heap)`.

2.2 Conteo de Tablas e Índices por Tabla

Indica la cantidad total de tablas y cuántos índices tiene cada una.

- **Vistas utilizadas:** `sys.tables`, `sys.schemas`, `sys.indexes`.
- **Lógica:** Conteo escalar de tablas y `LEFT JOIN` con `sys.indexes` agrupando por tabla.

2.3 Restricciones del Esquema

Enumera las restricciones de integridad declaradas.

- **Vistas utilizadas:** `sys.objects`, `sys.tables`, `sys.schemas`.
- **Clasificación:** Se filtran por `c.type`:

Código	Tipo de Restricción
PK	Clave Primaria
F	Clave Foránea
C	Check Constraint
UQ	Unicidad
D	Valor por Defecto

2.4 Columnas de cada Índice

Detalla qué columnas componen cada índice y su orden.

- **Vistas utilizadas:** `sys.indexes`, `sys.index_columns`, `sys.columns`.
- **Lógica:** Se usa `STRING_AGG` ordenando por `ic.key_ordinal`, indicando `ASC/DESC` según `ic.is_descending_key`.

2.5 Triggers del Esquema

Lista los disparadores asociados a las tablas.

- **Vistas utilizadas:** `sys.triggers`, `sys.trigger_events`, `sys.tables`.
- **Lógica:** Se clasifica el tipo (`AFTER` o `INSTEAD OF`) y el estado (habilitado/deshabilitado). Los eventos se agrupan con `STRING_AGG`.

2.6 Tamaño Físico por Tabla

Muestra el espacio en disco ocupado por cada tabla.

- **Vistas utilizadas:** `sys.tables`, `sys.dm_db_partition_stats`.
- **Fórmulas de Almacenamiento** (SQL Server almacena datos en páginas de 8 KB):

$$\text{Espacio de Datos (KB)} = \text{Páginas de Datos} \times 8 \quad (1)$$

$$\text{Espacio de Índices (KB)} = \text{Páginas de Índices} \times 8 \quad (2)$$

Donde las páginas de datos corresponden a registros con `index_id ∈ {0, 1}` (tabla base/Heap o índice clúster), y las páginas de índices a registros con `index_id > 1` (índices no agrupados).

2.7 Estimación del Tamaño de Registro

Estima cuántos bytes ocupa un registro completo de cada tabla en promedio.

- **Vistas utilizadas:** `sys.columns`, `sys.tables`.
- **Fórmula de Estimación:**

$$\text{Tamaño del Registro} = \sum (\text{Longitud de cada Columna}) \quad (3)$$

Se obtiene sumando los tamaños máximos (`max_length`) de todas las columnas de la tabla.

Ejemplo práctico: Una columna `VARCHAR(50)` aporta 50 bytes, mientras que una de tipo

NVARCHAR(50) aporta 100 bytes (debido a la codificación de doble byte de caracteres Unicode).

2.8 Tamaño de cada Columna según su Tipo de Dato

Desglosa el tipo de dato y tamaño físico de cada columna.

- **Vistas utilizadas:** `sys.columns`, `sys.types`.
- **Lógica:** Se reconstruye la definición SQL de la columna a partir de su tipo mediante sentencias condicionales (`varchar(n)`, `nvarchar(n/2)`, `decimal(p,s)`, etc.).

2.9 Factor de Bloqueo de Tablas e Índices

El **Factor de Bloqueo** indica cuántos registros caben como máximo en una única página física de 8 KB (8192 bytes).

- **Factor de Bloqueo de la Tabla:**

$$\text{Factor de Bloqueo}_{\text{Tabla}} = \left\lfloor \frac{8192}{\text{Tamaño del Registro}} \right\rfloor \quad (4)$$

- **Factor de Bloqueo del Índice** (basado solo en las columnas indexadas):

$$\text{Factor de Bloqueo}_{\text{Índice}} = \left\lfloor \frac{8192}{\text{Tamaño de la Clave}} \right\rfloor \quad (5)$$

Donde el *Tamaño de la Clave* es la suma de las longitudes de las columnas que forman parte de dicho índice.

2.10 Costo de Consultas de Igualdad

Estima el costo de ejecutar una consulta de tipo `WHERE columna = valor` en términos de accesos a páginas y tiempo.

2.10.1 1. Detección de Índices

Un índice es utilizable para búsquedas directas si la columna por la que se consulta es la primera clave registrada en la estructura (`key_ordinal = 1`).

2.10.2 2. Caso A: Escaneo Completo (Full Table Scan)

Sin un índice útil, SQL Server debe leer todas las páginas físicas de la tabla:

$$\text{Costo}_{\text{Scan}} = \text{Total de Páginas} \quad (6)$$

$$\text{Tiempo}_{\text{Scan}} = \frac{\text{Total de Páginas} \times 8 \text{ KB}}{17 \text{ MB/s}} \quad (7)$$

2.10.3 3. Caso B: Búsqueda por Índice (Index Seek)

Con un índice aplicable, se recorre su estructura en árbol B-Tree. La altura aproximada del árbol (h) se estima con un factor de ramificación (*fan-out*) promedio de 100:

$$\text{Altura del Árbol } (h) = \max(1, \lceil \log_{100}(\text{Total de Páginas}) \rceil) \quad (8)$$

El costo se calcula dependiendo del tipo de índice detectado:

- **Índice Clustered:** $\text{Costo}_{\text{Seek}} = h + 1$ (las hojas contienen directamente los datos).
- **Índice Non-Clustered:** $\text{Costo}_{\text{Seek}} = h + 2$ (requiere el recorrido y luego leer la página de datos o *lookup*).

Para tablas con pocas páginas, se limita el costo del Seek para que no supere al de un Scan completo:

$$\text{Costo Seek Final} = \min(\text{Costo}_{\text{Seek}}, \text{Costo}_{\text{Scan}}) \quad (9)$$

El tiempo estimado de búsqueda física se deduce homológamente:

$$\text{Tiempo}_{\text{Seek}} = \frac{\text{Costo Seek Final} \times 8 \text{ KB}}{17 \text{ MB/s}} \quad (10)$$

3. Instrucciones de Ejecución

3.1 Paso 1: Prerrequisitos

1. **Python 3.11+**. Verificar con:

```
python --version
```

2. **Microsoft SQL Server** activo y accesible.
3. **Driver ODBC**: ODBC Driver 17 for SQL Server (recomendado) u ODBC Driver 18.

3.2 Paso 2: Instalar Dependencias

Desde la raíz del proyecto (Fase-1):

```
pip install -r requirements.txt
```

Esto instala **Streamlit** (interfaz web), **pyodbc** (conexión ODBC) y **pandas** (manejo de datos).

3.3 Paso 3: Crear y Poblar la Base de Datos

El script `setup_db.py` ejecuta en orden los archivos SQL de la carpeta `scripts/`:

1. `create_repositorio_sqlserver.sql`: Crea la BD StreamUCV y el esquema streaming.
2. `create_tables_sqlserver.sql`: Crea tablas, claves, restricciones e índices.
3. `insert_tables_sqlserver.sql`: Carga los datos de prueba.

```
python setup_db.py
```

*Nota: Si falla la conexión, verifique que **SERVER** en `app/config.py` coincida con su instancia (ej. `localhost\SQLEXPRESS`).*

3.4 Paso 4: Configurar la Conexión

Edite `app/config.py` según su entorno:

```
app/config.py

SERVER = "localhost\\SQLEXPRESS"
DATABASE = "StreamUCV"
USERNAME = "" # Vacio para Windows Auth
PASSWORD = ""
DRIVER = "ODBC Driver 17 for SQL Server"
```

Si `USERNAME` y `PASSWORD` quedan vacíos, se usa `Trusted_Connection=yes` (autenticación de Windows).

3.5 Paso 5: Ejecutar la Aplicación

```
Terminal

streamlit run main.py
```

Se abrirá automáticamente en <http://localhost:8501>.

3.6 Paso 6: Navegación

1. **Barra Lateral:** Permite modificar los parámetros de conexión en caliente y probar la conexión sin reiniciar.
2. **Pestañas principales:**
 - **Estructura Lógica:** Tablas e índices (R1), conteos (R2), restricciones (R3), detalle de índices (R4).
 - **Disparadores:** Triggers del esquema (R5).
 - **Almacenamiento:** Tamaño por tabla (R6), tamaño de registro (R7), tamaño por columna (R8).
 - **Factor de Bloqueo:** Registros por página de tablas e índices (R9).
 - **Costo de Consulta:** Selección de tabla/columna para comparar Full Scan vs. Index Seek (R10).